

Module 6

IEEE 1800-2017 (Current SystemVerilog)

Learning objectives

- Master the current SystemVerilog standard (IEEE 1800-2017): unified LRM, merged Verilog and SystemV...

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["IEEE 1800-2017 Context"]

T2["Verilog as a Subset of 1800-"]

T1 --> T2

T3["Design Constructs in 1800-20"]

T2 --> T3

Master the current SystemVerilog standard (IEEE 1800-2017): unified LRM, merged Verilog and SystemVerilog, and design/verification overview.

Overview

- This module covers IEEE Std 1800-2017, the current SystemVerilog standard. In 1800-2017, the Verilo...

Syllabus topics

1. IEEE 1800-2017 Context (1/2)

- Before 2017: IEEE 1364 defined Verilog; IEEE 1800 defined SystemVerilog (extending 1364). Two documents.
- 1800-2017: One LRM (IEEE 1800-2017) defines both the Verilog subset and SystemVerilog. IEEE 1364 is obsolete.
- Implication: When you say "IEEE 1800-2017," you are referring to the full language (Verilog + SystemVerilog).
- Structure: Single LRM; consistent numbering and cross-references; Verilog and SystemVerilog in one document.
- Clarifications: Scheduling, generate, interfaces, assertions, coverage, DPI, and many other areas redefined.
- Corrections: Errata and ambiguous cases addressed.

1. IEEE 1800-2017 Context (2/2)

- Additions: Any new or clarified features in the 2017 edition (check LRM for specifics; often increm...
- No separate 1364: For new work, use 1800-2017; legacy "1364-only" is the subset of 1800 that omits...
- RTL and testbenches written for 1364-1995 through 1800-2012 remain valid in 1800-2017 when they use...
- Adopt 2017 as the reference standard for new projects unless a project or tool explicitly targets a...

2. Verilog as a Subset of 1800-2017 (1/2)

- Modules, ports: ANSI or non-ANSI (1364-1995 style);
wire, reg.
- Procedural blocks: always, initial; always @*
(2001); explicit sensitivity (1995).
- No SystemVerilog-only: No logic,
always_comb/always_ff, interfaces, packages,
unique/priority case...
- Use: When the flow or legacy code requires "Verilog
only" (e.g. synthesis or IP that does not accep...
- logic, always_comb, always_ff, always_latch.
- Interfaces and modports.

2. Verilog as a Subset of 1800-2017 (2/2)

- Packages and import.
- unique case, priority case.
- 2-state types (bit, int, etc.) where useful.
- Use: When the flow supports SystemVerilog; preferred for new RTL for clarity and tool checking.
- Verilog-only (1364 subset): Maximum portability; use when tools or IP are Verilog-only.
- SystemVerilog design (1800 subset): Better expressiveness and checking; use when the flow supports...

3. Design Constructs in 1800-2017 (Recap and Clarifications) (1/2)

- Rule: logic must have exactly one driver (one assign, one always block, or one port connection).
- 2017: Semantics of multiple drivers, resolution, and connection rules are clarified in the LRM.
- Intent: Combinational, sequential (flip-flop), or latch; tools can check and warn.
- 2017: Execution order, sensitivity, and interaction with other processes are clarified.
- Use: Bundle signals; modports define direction per role.
- 2017: Interface connection, parameterization, and modport rules are refined.

3. Design Constructs in 1800-2017 (Recap and Clarifications) (2/2)

- Use: Shared types, constants, functions; explicit namespace.
- 2017: Import scope, search order, and compilation-unit interaction are clarified.
- Semantics: At most one match (unique) or first match wins (priority); runtime checks possible.
- 2017: Case statement semantics and synthesis mapping are clarified.

4. Assertions and Verification in 1800-2017 (Context) (1/2)

- Immediate: ``assert (condition);`` — executed when reached; useful for invariants in RTL.
- Concurrent: ``assert property (...);`` — temporal; used in verification and bind files.
- 2017: Property syntax, scheduling, and binding are clarified.
- Checker: Reusable block of assertions/assumptions/coverage (from 2012); instantiated or bound.
- 2017: Checker semantics and instantiation are clarified.
- Covergroups, coverpoints: Defined in 1800; used in verification.

4. Assertions and Verification in 1800-2017 (Context) (2/2)

- 2017: Coverage options and semantics are refined.

5. Synthesis and Tool Support (1800-2017) (1/2)

- Generally supported: logic, always_comb, always_ff, interfaces, packages, generate, parameter, loca...
- Often not supported or tool-specific: Classes, full SVA, dynamic arrays/queues in RTL, virtual inte...
- Check tool docs: Each vendor defines the exact synthesizable subset for 1800-2017.
- 1800-2017: Most commercial and several open-source simulators support a large subset of 1800-2017.
- Revisions: Tools may support 2005, 2009, 2012, or 2017; choose project standard to match tool and I...
- Lint: Many tools check 1800-2017 syntax and common RTL rules (e.g. single driver, blocking/nonblock...

5. Synthesis and Tool Support (1800-2017) (2/2)

- Formal: Assertions and checkers are used with formal tools; 2017 clarifies semantics for consistenc...

6. Version Timeline Summary (1995 → 2017)

- See docs/MODULE6.md

Hands-on examples

Module 6 toolchain check

```
$ command -v iverilog >/dev/null && echo "Toolchain ready: $(iverilog  
-V 2>&1 | head -1)"
```

Terminal: module6_check

Toolchain ready: Icarus Verilog version 12.0 (stable) ()

Demo: Verilog vs SystemVerilog Subset

```
$ cd module6/examples/subsets && make clean && make run
```

Terminal: ex01_subsets

```
rm -f top
iverilog -g2012 -o top verilog_vs_sv.sv
vvp top
1800-2017: Verilog subset vs SystemVerilog subset
  sel=0 y1364=0 y1800=0
  sel=1 y1364=1 y1800=1
  Same behavior; 1800-2017 LRM defines both.
verilog_vs_sv.sv:43: $finish called at 20 (1s)
```

Demo: Design Constructs Recap

```
$ cd module6/examples/design_recap && make clean && make run
```

Terminal: ex02_design_recap

```
rm -f top
iverilog -g2012 -o top design_recap.sv
vvp top
1800-2017 design_recap: logic, always_comb, always_ff, package, unique case
count= 4 mux_sel=10 y= 4
design_recap.sv:68: $finish called at 60 (1s)
design_recap.sv:36: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
```

Demo: Immediate Assertion in RTL

```
$ cd module6/examples/assertions && make clean && make run
```

Terminal: ex03_assertions

```
rm -f top
iverilog -g2012 -o top immediate_in_rtl.v
vvp top
1800-2017: Immediate assertion in RTL (one-hot invariant)
  req=001 grant=001
  req=110 grant=010 (priority)
  req=000 grant=000
  Assertion passes when grant is one-hot or zero.
immediate_in_rtl.v:36: $finish called at 30 (1s)
immediate_in_rtl.v:11: sorry: constant selects in always_* processes are not currently supported
immediate_in_rtl.v:11: sorry: constant selects in always_* processes are not currently supported
immediate_in_rtl.v:11: sorry: constant selects in always_* processes are not currently supported
immediate_in_rtl.v:21: warning: System task ($error) cannot be synthesized in an always_comb process
```

Demo: Version Comparison

```
$ cd module6/examples/version_summary && make clean && make run
```

Terminal: ex04_version_summary

```
rm -f top
iverilog -g2012 -o top version_summary.sv
vvp top
1800-2017: Version timeline (1364-1995 -> 1800-2017)
  1364-1995: wire/reg, always, initial
  1364-2001: ANSI ports, always @*, generate
  1364-2005: last Verilog-only
  1800-2005: logic, always_comb/always_ff, interfaces, packages
  1800-2009/2012: inside/==?, checkers, array methods
  1800-2017: unified LRM (Verilog + SystemVerilog in one standard)
sel=11 y=08
version_summary.sv:41: $finish called at 10 (1s)
version_summary.sv:15: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
```

Demo: Migration

```
$ cd module6/examples/migration && make clean && make run
```

Terminal: ex05_migration

```
rm -f top
iverilog -g2012 -o top migration.sv
vvp top
1800-2017: Migration 1364 -> 1800 design subset
  sel=10 y1364=04 y1800=04 (same)
  Steps: wire/reg->logic, always @*->always_comb, add unique case
migration.sv:52: $finish called at 10 (1s)
migration.sv:30: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
```

Demo: Logic Single Driver

```
$ cd module6/examples/logic_single_driver && make clean && make run
```

Terminal: ex06_logic_single_driver

```
rm -f top
iverilog -g2012 -o top logic_single_driver.sv
vvp top
1800-2017: logic single driver (2017 clarified)
  comb: a= 10 b= 20 sum= 30 eq=0
  seq:  d= 55 q= 55
  Each logic has one driver: assign, always_comb, or always_ff.
logic_single_driver.sv:50: $finish called at 50 (1s)
```

Demo: Priority Case

```
$ cd module6/examples/priority_case && make clean && make run
```

Terminal: ex07_priority_case

```
rm -f top
iverilog -g2012 -o top priority_case.sv
vvp top
1800-2017: priority case (first match wins)
  in=0100 enc=10 valid=1 (bit 2)
  in=0011 enc=00 valid=1 (bit 0 wins)
  in=0000 enc=00 valid=0 (none)
priority_case.sv:36: $finish called at 30 (1s)
priority_case.sv:11: sorry: constant selects in always_* processes are not currently supported (
priority_case.sv:11: sorry: constant selects in always_* processes are not currently supported (
priority_case.sv:11: sorry: constant selects in always_* processes are not currently supported (
priority_case.sv:11: sorry: constant selects in always_* processes are not currently supported (
```

Demo: Package and Import

```
$ cd module6/examples/package_import && make clean && make run
```

Terminal: ex08_package_import

```
rm -f top
iverilog -g2012 -o top package_import.sv
vvp top
1800-2017: package and import (2017 clarified)
  a= 40 b= 60 sum=100
  Shared types/params in package; import where needed.
package_import.sv:33: $finish called at 10 (1s)
```

Demo: Unique Case

```
$ cd module6/examples/unique_case && make clean && make run
```

Terminal: ex09_unique_case

```
rm -f top
iverilog -g2012 -o top unique_case.sv
vvp top
1800-2017: unique case (at most one match)
  sel=00 y=0001
  sel=10 y=0100
  sel=11 y=1000
  unique case: no overlap; default avoids latch.
unique_case.sv:34: $finish called at 30 (1s)
unique_case.sv:12: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
```

Practice & assessment

Exercises

- Subset Comparison
- 1800-2017 Design Checklist
- Immediate Assertion
- Version Table
- Migration Plan

Exercises

- Assuming 1364 Is Still a Separate Standard
- Mixing Revisions in One Project
- Ignoring Tool Support
- Assertions in Synthesis
- Over-Migrating Legacy Code

Summary & next steps

- Master the current SystemVerilog standard (IEEE 1800-2017): unified LRM, merged Verilog and SystemV...
- Complete module6/CHECKLIST.md
- Run `./scripts/module6.sh --check`
- Next: Next module in course